

IA Aplicada com LLMs

Aula 01 — Grandes modelos de linguagem, tokens e tokenização

Onde paramos

A Parte 1 terminou aqui:

```
... blocos decoder-only → projeção no vocabulário →  $P(\text{próximo token})$  → sorteio
```

Duas perguntas ficaram abertas:

1. **O que é esse vocabulário?** O que é, exatamente, um "token"?
2. **O que "grande" significa** em "grande modelo de linguagem"?

E uma terceira, prometida: **como a distribuição vira texto.**

Roteiro

1. **O que é um modelo de linguagem** — e por que os n-gramas falharam
2. **O que "grande" significa** — e as duas consequências que confundem todo mundo
3. **Decodificação** — logits, `temperature`, `top_p`
4. **Tokens e tokenização** — BPE, custo, e por que o modelo não conta letras

Objetivos

- Definir modelo de linguagem e explicar a **regra da cadeia**
- Justificar por que **alucinação é o comportamento esperado**
- Explicar por que o modelo **não tem memória** entre chamadas
- Escolher `temperature` e `top_p` por tipo de tarefa
- Executar o **BPE** à mão e medir tokens de verdade
- Identificar as falhas que são **consequência da tokenização**

Parte 1

O que é um modelo de linguagem

A definição

Um **modelo de linguagem** atribui **probabilidade a sequências de texto**.

Nada além disso.

$$P(w_1, w_2, \dots, w_n)$$

- alta para "o gato subiu no telhado"
- baixa para "telhado no subiu gato o"
- baixíssima para "o gato subiu no fotossíntese"

A regra da cadeia

Estimar a sequência inteira parece intratável. Mas:

$$P(w_1, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_1, \dots, w_{i-1})$$

Modelar a linguagem inteira se reduz a prever o próximo token.

Cada fator é exatamente a saída do Transformer da Parte 1.

Três ângulos, uma decisão

Conceito	O que é
Objetivo de treino	maximizar $P(w_i \mid \text{contexto})$ — entropia cruzada
Arquitetura	decoder-only com máscara causal: só olha o passado
Geração	amostrar w_i , anexar, repetir

Não são três escolhas que por acaso combinam.

São **a mesma escolha** vista de três lados.

Antes das redes: n-gramas

Hipótese de Markov: só as $n - 1$ palavras anteriores importam. E então **conte**:

$$P(w_i \mid w_{i-1}) = \frac{\text{contagem}(w_{i-1}, w_i)}{\text{contagem}(w_{i-1})}$$

```
o gato subiu no telhado  
o gato dormiu no sofá  
o cachorro dormiu no tapete
```

Depois de "o": gato 2×, cachorro 1× $\rightarrow P(\text{gato} \mid \text{o}) = 2/3$

Estado da arte em fala e tradução por **décadas**.

Os dois problemas fatais

1. Esparsidade

Sequência não vista → probabilidade **zero** → zera a frase inteira.

"o cachorro subiu no telhado" é português perfeito e tem probabilidade 0.

2. Nenhuma generalização ← o problema profundo

"gato" e "cachorro" são **símbolos distintos**, sem relação.

Aprender que gatos dormem não ensina nada sobre cachorros.

E o contexto é curto por construção: 5-gramas **não podem** ligar "chave" a "estava" com 17 palavras no meio.

O que a rede mudou

Representação distribuída

Cada token vira um **vetor**. Sentidos parecidos → vetores parecidos → o aprendizado **transfere**.

Contexto longo

Primeiro RNN, depois — de verdade — a atenção.

Bônus: o conhecimento fica **comprimido nos pesos**, não numa tabela de contagens.

Guarde a palavra **compressão**.

Parte 2

O que "grande" significa

Os três eixos

Eixo	Ordem de grandeza hoje
Parâmetros	10^9 a 10^{12}
Dados de treino (tokens)	10^{12} a 10^{13}
Computação (FLOPs)	10^{22} a 10^{25}

Os três crescem juntos — e o Chinchilla deu a proporção: **~20 tokens por parâmetro**.

A escala, em perspectiva

Modelo	Parâmetros	Contexto
n-gramas	tabela de contagens	2–5 tokens
LSTM (2015)	~10 milhões	~100 tokens
GPT-2 (2019)	1,5 bilhões	1.024
GPT-3 (2020)	175 bilhões	2.048
Mistral 7B (2023)	7,2 bilhões	8.192
Fronteira (2024–)	não divulgado	10^5 – 10^6

A mudança qualitativa

Não foi só "o mesmo modelo, maior". Apareceu o que ninguém programou:

- **In-context learning** — descrever a tarefa no prompt e ela é executada
- **Seguir instruções**, incluindo formato ("responda em JSON")
- **Raciocínio em passos**, quando induzido
- **Transferência** entre idiomas e domínios

Quando você escreve "Resuma em uma sentença", **não existe** um modelo de sumarização. Existe um modelo que continua o texto de um jeito que é um resumo.

Cuidado com a palavra "emergente"

Wei et al. (2022): capacidades **emergentes** — ausentes nos pequenos, presentes nos grandes.

Schaeffer et al. (2023): o "salto" é em boa parte **artefato de métrica descontínua**.
Com métrica contínua, a melhora é gradual.

Não há controvérsia sobre o fato de modelos grandes fazerem o que pequenos não fazem.

A controvérsia é se há salto ou curva suave.

O que o modelo realmente guardou

Um LLM **não** é um banco de dados de fatos.

É **compressão com perdas da distribuição estatística** do texto de treino.

Pesos de um modelo 7B	~14 GB
Texto de treino	dezenas de TB
Razão de compressão	milhares para um

Nada é armazenado literalmente. Sobrevivem os **padrões** que ajudam a prever o próximo token.

Consequência 1: alucinação

O objetivo de treino é **plausibilidade estatística**, não verdade.

Em nenhum momento o modelo foi otimizado para dizer a verdade ou admitir que não sabe.

Pergunte o título do 3º artigo de um pesquisador obscuro:

- ele não tem tabela para consultar
- tem uma distribuição sobre "como títulos dessa área se parecem"
- **gera um título plausível**, com a mesma confiança de quando acerta

Não existe sinal interno separando "recuperado" de "inventado".

Logo: não é um bug de prompt

Alucinação é **característica estrutural** do objetivo de treino.

Você não elimina — você **arquiteta em volta**:

Estratégia	Módulo do curso
Dar o texto-fonte no contexto	RAG
Dar ferramentas para consultar a verdade	agentes
Verificar a saída	evals

Consequência 2: sem memória

O modelo é uma **função pura**. Pesos **congelados**. Sem estado entre chamadas.

Então por que o chatbot parece lembrar?

```
messages.append({"role": "user", "content": user_input})  
response = client.chat.completions.create(model=..., messages=messages)  
messages.append({"role": "assistant", "content": answer})
```

A lista **inteira** é reenviada a cada turno.

A memória está no seu programa, não no modelo.

Três consequências práticas

1. **O custo cresce** — o turno 20 processa muito mais tokens que o turno 1.
A soma cresce com n^2 .
2. **A conversa estoura a janela** — e você decide o que descartar ou resumir.
3. **Gerenciar contexto é trabalho de engenharia** — o seu.
É o que hoje se chama *context engineering*.

Parte 3

Da distribuição ao texto

Logits e softmax

A última camada produz números reais sem restrição — um por token do vocabulário.

São os **logits**. Não somam 1, podem ser negativos.

$$P(\text{token}_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Para um vocabulário de 32 mil tokens: **32 mil logits por posição**.

Temperatura

Uma única modificação: divida os logits por T antes de exponenciar.

$$P(\text{token}_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- $T < 1 \rightarrow$ **amplifica** as diferenças: distribuição pontuda
- $T > 1 \rightarrow$ **encolhe** as diferenças: distribuição achatada
- $T \rightarrow 0 \rightarrow$ `argmax` , determinístico

Temperatura, em números

Logits [3.2, 2.1, 1.0, 0.2] para "O céu estava..."

T	azul	claro	cinza	roxo
0.1	100.0%	0.0%	0.0%	0.0%
0.5	88.8%	9.8%	1.1%	0.2%
1.0	67.0%	22.3%	7.4%	3.3%
2.0	46.9%	27.0%	15.6%	10.5%

Os logits não mudaram. A temperatura não altera o que o modelo "pensa" — só o quanto ele arrisca ao escolher.

top_k e top_p

Temperatura mexe na distribuição inteira, inclusive na cauda de tokens absurdos.

top_k — mantém os k mais prováveis

→ k fixo é ruim: às vezes há 2 continuações razoáveis, às vezes 200

top_p (*nucleus*) — mantém tokens até a soma acumulada atingir p

→ com `top_p=0.9`, o corte **se adapta** à confiança do modelo

Recomendação: **ajuste um, não os dois.**

Qual valor usar

Tarefa	temperature
Extração, classificação, JSON	0 – 0,3
Resposta factual, código	0,2 – 0,5
Redação, explicação	0,7 – 1,0
Criação, brainstorm	1,0 – 1,5

Pedir um JSON de cadastro é caso de **temperatura baixa** — variação ali é defeito, não criatividade.

`temperature=0` $\neq$ determinismo

A escolha *dentro de uma inferência* é determinística. O resultado entre requisições **ainda varia**:

- requisições são agrupadas em **lotes** de tamanho variável
- soma de ponto flutuante **não é associativa**
- modelos *mixture-of-experts* roteiam de forma sensível ao lote
- o provedor atualiza o modelo por trás de `...-latest`

Não escreva testes que exijam string idêntica. Teste propriedades: "é JSON válido", "tem os 3 campos".

Parte 4

Tokens e tokenização

O modelo lê inteiros

```
"Por que o céu é azul?"  
  ↓ tokenizador (encode)  
[8582, 1744, 297, 12088, 1289, 21361, 30]  
  ↓ embeddings + posição + N blocos  
distribuição sobre 32 mil tokens → sorteio → id  
  ↓ tokenizador (decode)  
"Porque"
```

Estruturas de Dados: o vocabulário é uma **tabela hash** `string → int`, mais o vetor inverso. Busca $O(1)$.

Determinístico e reversível: `decode(encode(t)) == t`.

Por que não caractere nem palavra

Caractere — vocabulário de ~100, nunca há palavra desconhecida

→ mas sequências **5× mais longas**, e a atenção é **quadrática**: custo $\times 25$

Palavra — sequências curtas, unidades com significado

→ mas vocabulário **aberto**: sempre aparece palavra nova → `<UNK>`

→ e "caminhar/caminhava/caminharíamos" são entradas **sem relação**

Subpalavra: o meio-termo que venceu

Estratégia	Vocabulário	Comprimento	Desconhecida
Caractere	~100	péssimo	impossível
Palavra	infinito	ótimo	<UNK>
Subpalavra	32k – 130k	bom	impossível

Palavras frequentes ficam inteiras; raras se quebram em pedaços reaproveitáveis.

E o tokenizador **aprende** onde cortar.

BPE: a regra inteira

Byte Pair Encoding (Sennrich et al., 2016), vindo da compressão de dados:

Comece com caracteres. Repita: funda o **par adjacente mais frequente** num símbolo novo. Pare no tamanho de vocabulário desejado.

A lista ordenada de fusões é o tokenizador.

BPE rodando

Corpus: baixo baixa baixinho caixa caixote caixinha peixe peixinho ...

```
1. funde 'i' + 'x'      (10x) -> 'ix'
2. funde 'a' + 'ix'     ( 7x) -> 'aix'
3. funde 'a' + '_'      ( 6x) -> 'a_'
4. funde 'b' + 'aix'    ( 4x) -> 'baix'
6. funde 'i' + 'n'      ( 3x) -> 'in'
7. funde 'in' + 'h'     ( 3x) -> 'inh'
8. funde 'c' + 'aix'    ( 3x) -> 'caix'
```

```
caixinha_ -> ['caix', 'inh', 'a_']
```

O que o BPE descobriu sozinho

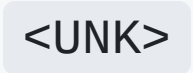
```
caixinha_ -> ['caix', 'inh', 'a_']
```

Pedaço	O que é
caix	o radical de caixa/caixote/caixinha
inh	o sufixo de diminutivo
a_ , o_	as terminações de gênero

Morfologia redescoberta por pura contagem de frequência.

E é isso que permite generalizar: "caixinha" e "caixote" compartilham **caix** .

Variantes que você vai encontrar

Variante	Onde	Diferença
BPE	GPT, Llama, Mistral	funde o par mais frequente
WordPiece	BERT	funde o par que mais aumenta a verossimilhança
SentencePiece	Llama, T5	opera no texto cru; espaço é caractere ()
Byte-level BPE	GPT-2 em diante	opera sobre bytes — nunca precisa de 

Com 256 bytes como base, **qualquer** entrada é tokenizável: emoji, ideograma, binário colado por engano.

Medindo de verdade

tiktoken já está no requirements.txt do curso:

```
morango          2 tokens -> ['mor', 'ango']
strawberry       3 tokens -> ['str', 'aw', 'berry']
inteligência     3 tokens -> ['int', 'elig', 'ência']
paralelepípedo   5 tokens -> ['par', 'ale', 'lep', 'í', 'pedo']
tokenização      2 tokens -> ['token', 'ização']
```

tokenização → radical técnico + sufixo. paralelepípedo → 5 pedaços sem sentido.

Palavra longa e rara é onde a fragmentação aparece.

Português custa mais — quanto?

Mesmo parágrafo, medido:

inglês	229 caracteres	41 tokens	5.59 chars/token
português	237 caracteres	60 tokens	3.95 chars/token

+46% de tokens para o mesmo conteúdo

46% mais custo, mais ocupação de janela, mais latência.

Mas não generalize demais

Inglês	tokens	Português	tokens
cat	1	gato	2
dog	1	cachorro	3
The book is on the table	6	O livro está sobre a mesa	6

A última linha **empata**.

A regra não é "português sempre custa mais por palavra" — é que palavras portuguesas **fragmentam mais**, e o efeito **se acumula** em texto longo.

Meça o seu caso.

A contagem que vale é a da API

```
response = client.chat.completions.create(...)

response.usage.prompt_tokens      # entrada
response.usage.completion_tokens # saída
response.usage.total_tokens
```

Fonte de verdade: o tokenizador do próprio modelo, **incluindo** os tokens especiais que marcam `system / user / assistant` — que também são cobrados.

Onde a tokenização vaza

Contar letras — "quantos R em `strawberry` ?" O modelo vê `['str', 'aw', 'berry']`. A informação sobre caracteres **não está** na entrada.

Aritmética — números quebram de forma inconsistente. O algoritmo de soma opera em **dígitos alinhados**; o modelo opera em pedaços arbitrários.

Inverter string, contar sílabas, rimar — todas em nível de caractere.

Quando a tarefa é sobre os **caracteres**, e não sobre o significado: desconfie e **dê uma ferramenta**.

As duas contas que você sempre faz

Cabe na janela?

system + histórico + documentos (RAG) + **espaço para a resposta**

← esquecer o último é erro clássico

Quanto custa?

$$\text{custo} = \frac{\text{tokens}_{\text{in}}}{10^6} \times p_{\text{in}} + \frac{\text{tokens}_{\text{out}}}{10^6} \times p_{\text{out}}$$

Saída costuma custar mais que entrada. Consulte o preço vigente — ele muda.

Definição operacional

Um **LLM** é uma **função sem estado** que recebe tokens e devolve uma **distribuição sobre o próximo token**. Transformer decoder-only, bilhões de parâmetros, treinado para minimizar surpresa sobre trilhões de tokens e depois alinhado. Texto é gerado aplicando a função repetidamente.

```
tokens —> [ LLM: função pura, pesos congelados ] —> distribuição
                                     |
contexto + token <— sorteio (temperature) <—
```

O que ele NÃO é

Não é	Por quê	Solução
Banco de fatos	compressão com perdas; alucina	RAG
Calculadora	opera em tokens, não dígitos	ferramentas
Sistema com memória	função pura, sem estado	memória externa
Agente	só produz o próximo token	o laço + ferramentas

A última linha é a Parte 3.

Síntese — o modelo

- **Modelo de linguagem** atribui probabilidade a sequências; pela **regra da cadeia**, isso vira prever o próximo token
- **n-gramas** falharam por esparsidade e por **não generalizar**
- "**Grande**" = parâmetros + dados + computação, e o que veio de novo: **in-context learning**
- LLM é **compressão com perdas** → **alucinação é esperada**, e se contorna com arquitetura
- **Função pura sem estado** → a memória é do seu programa, e o custo do chat cresce com n^2

Síntese — tokens

- **Decodificação:** logits $\rightarrow$ softmax $\rightarrow$ sorteio. `temperature` reformata; `top_p` corta a cauda. Ajuste **um**
- `temperature=0` **não** garante saída idêntica
- **Token** é a unidade real de custo, janela e cobrança
- **Subpalavra** venceu: caractere estoura o custo quadrático, palavra estoura o vocabulário
- **BPE** funde o par mais frequente e **redescobre morfologia** sozinho
- **Português ~46% mais tokens** (medido) — mas meça o seu caso
- Falhas com **letras e aritmética** vêm da tokenização $\rightarrow$ **dê uma ferramenta**

Para ir além

Papers

- Sennrich et al. — *Neural MT of Rare Words with Subword Units* (2016) — o BPE
- Holtzman et al. — *The Curious Case of Neural Text Degeneration* (2019) — o `top_p`
- Wei et al. (2022) e Schaeffer et al. (2023) — capacidades emergentes, leia os dois

Didático

- Karpathy — *Let's build the GPT Tokenizer* (YouTube)
- Jurafsky & Martin — *Speech and Language Processing*, cap. 3 (n-gramas)
- `platform.openai.com/tokenizer` — cole texto e veja a segmentação

Nesta aula: `notas-de-aula/02-llms-tokens-e-tokenizacao.md`